

Data Structures - Practice Questions

Topics: Arrays, Stack, Queue, Linked List

Q1. What is a Data Structure? Give 3 real-life examples.

A Data Structure is a way of organizing, storing, and managing data in a computer so it can be accessed and modified efficiently.

Real-life examples:

1. Stack — A stack of plates (last placed, first removed).
2. Queue — People standing in a ticket counter line (first come, first served).
3. Array — A row of lockers in a school, each with a number (index).

Q2. Differentiate between Linear and Non-linear Data Structures.

Linear Data Structures	Non-linear Data Structures
Elements arranged in sequence	Elements arranged hierarchically
Traversed in a single run	Multiple levels of traversal
Examples: Array, Stack, Queue	Examples: Tree, Graph
Memory usage is less flexible	Better for complex relationships

Q3. Explain the internal structure of an Array with a memory diagram.

An array stores elements in contiguous (sequential) memory locations. Each element is accessed using an index starting from 0. If base address is 1000 and each int takes 2 bytes:

Index:	[0]	[1]	[2]	[3]	[4]
Value:	10	20	30	40	50
Address:	1000	1002	1004	1006	1008

Formula: Address of `arr[i]` = Base Address + $i \times \text{size_of}(\text{datatype})$

Example: `arr[3]` = $1000 + 3 \times 2 = 1006$

Q4. Write the algorithm and C program for Array Insertion.

Algorithm:

- Step 1: Start
- Step 2: Input array `arr[]`, size `n`, element `x`, position `pos`
- Step 3: Shift elements right from index `n-1` down to `pos`
- Step 4: Insert `x` at `arr[pos]`
- Step 5: Increment `n`
- Step 6: Display updated array
- Step 7: Stop

C Program:

```
#include <stdio.h>

int main() {
    int arr[50], n, x, pos, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for (i = 0; i < n; i++) scanf("%d", &arr[i]);
```

```

printf("Enter position to insert (0-indexed): ");
scanf("%d", &pos);
printf("Enter element to insert: ");
scanf("%d", &x);
for (i = n; i > pos; i--)
    arr[i] = arr[i - 1];
arr[pos] = x;
n++;
printf("Array after insertion:\n");
for (i = 0; i < n; i++) printf("%d ", arr[i]);
printf("\n");
return 0;
}

```

Q5. Explain Stack using LIFO concept with a diagram.

Stack follows LIFO — Last In, First Out. The last element inserted is the first one to be removed. All insertions (Push) and deletions (Pop) happen at the TOP of the stack.

Push 10 → Push 20 → Push 30

```

| 30 | ← TOP (last in, first out)
| 20 |
| 10 |
|____|

```

Pop → removes 30 first (LIFO)

Operations:

Push(x) → Insert at top
 Pop() → Remove from top
 Peek() → View top without removing

Q6. What is the difference between Stack and Queue?

Feature	Stack	Queue
Principle	LIFO (Last In First Out)	FIFO (First In First Out)
Insertion	At TOP	At REAR
Deletion	From TOP	From FRONT
Example	Browser Back button	Ticket counter line
Operations	Push, Pop, Peek	Enqueue, Dequeue, Front
Overflow	Top == MAX-1	Rear == MAX-1
Underflow	Top == -1	Front == -1 or Front > Rear

Q7. Write C code for Enqueue and Dequeue operations.

```

#include <stdio.h>
#define MAX 5

int queue[MAX], front = -1, rear = -1;

void enqueue(int x) {
    if (rear == MAX - 1) { printf("Queue Overflow\n"); return; }
    if (front == -1) front = 0;
    queue[++rear] = x;
}

```

```

        printf("%d enqueued\n", x);
    }

void dequeue() {
    if (front == -1 || front > rear) { printf("Queue Underflow\n"); return; }
    printf("%d dequeued\n", queue[front++]);
    if (front > rear) front = rear = -1;
}

void display() {
    if (front == -1) { printf("Queue is empty\n"); return; }
    printf("Queue: ");
    for (int i = front; i <= rear; i++) printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    enqueue(10); enqueue(20); enqueue(30);
    display();
    dequeue();
    display();
    return 0;
}

```

Q8. Explain Singly Linked List with a node diagram.

A Singly Linked List is a dynamic data structure where each node contains data and a pointer to the next node. The last node points to NULL.

Node Structure:

```

+-----+-----+
| Data | Next |---->
+-----+-----+

```

Linked List with 3 nodes:

HEAD

```

|
v
+---+---+   +---+---+   +---+---+
| 10 | *--->| 20 | *--->| 30 | NULL |
+---+---+   +---+---+   +---+---+

```

C Structure:

```

struct Node {
    int data;
    struct Node* next;
};

```